

Einsendeaufgabe REF

Ausgewählte Refactorings aus dem Fowler-Katalog

Ich finde **Extract Method** besonders wichtig. Dabei wird ein längerer Codeabschnitt in eine eigene Methode ausgelagert. Dadurch wird der Code übersichtlicher und leichter zu verstehen. Außerdem kann man die neue Methode später einfacher testen oder wiederverwenden.

Auch **Replace Nested Conditional with Guard Clauses** finde ich sinnvoll. Dabei werden viele verschachtelte `if`-Bedingungen durch einfache Prüfungen am Anfang ersetzt. So sieht man schneller, was in Sonderfällen passiert, und der normale Ablauf der Methode wird klarer.

Schwieriger zu verstehen finde ich **Replace Superclass with Delegate**. Dabei wird Vererbung durch Delegation ersetzt. Die Idee kann sinnvoll sein, aber ich finde sie nicht sofort leicht verständlich, weil man dafür Vererbung und Delegation gut unterscheiden muss. Für kleinere Projekte wirkt dieses Refactoring auf mich eher kompliziert.

Verwendete IDE

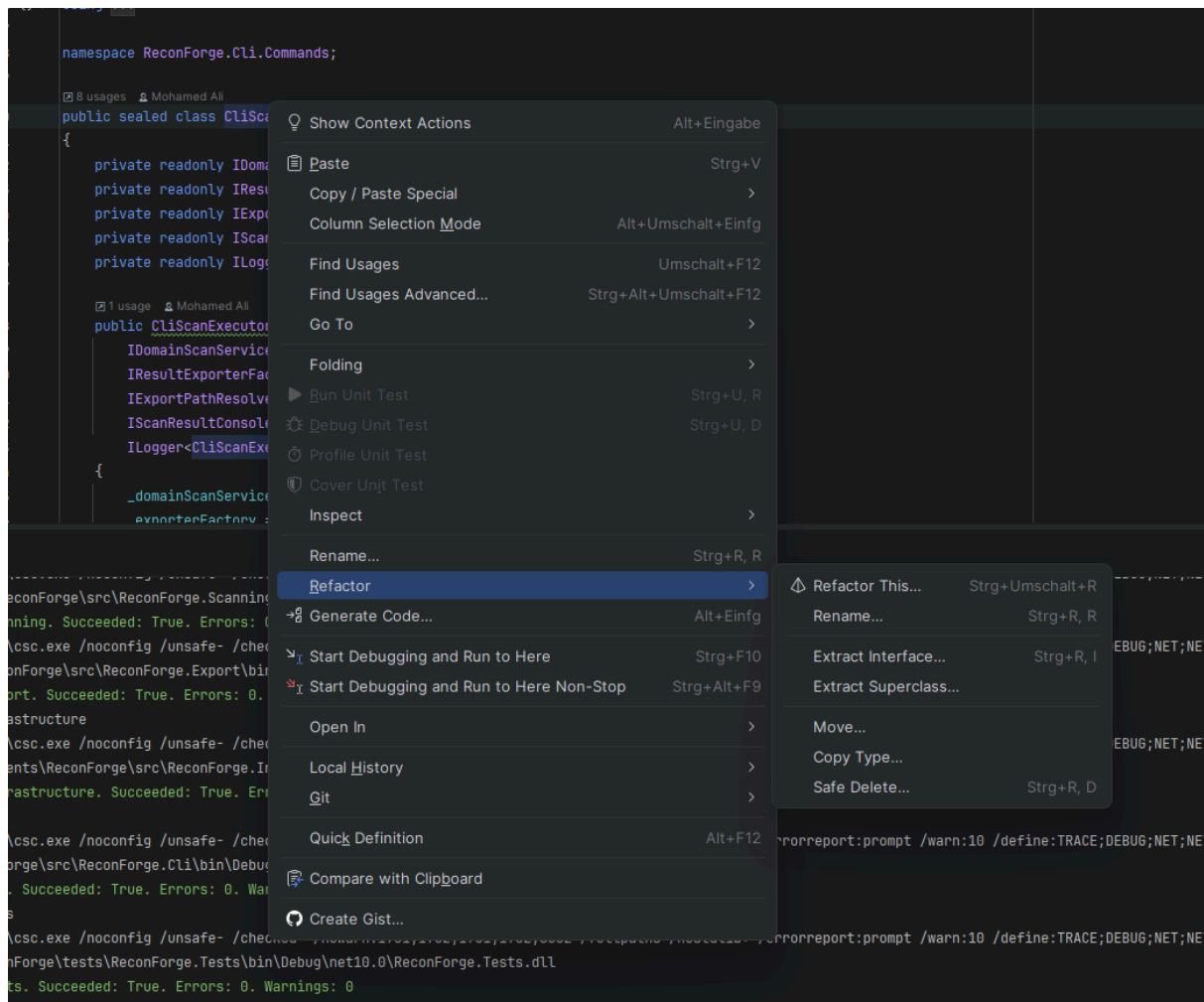
Für die Entwicklung nutze ich **JetBrains Rider**. Rider unterstützt verschiedene Refactorings für C#- und .NET-Projekte.

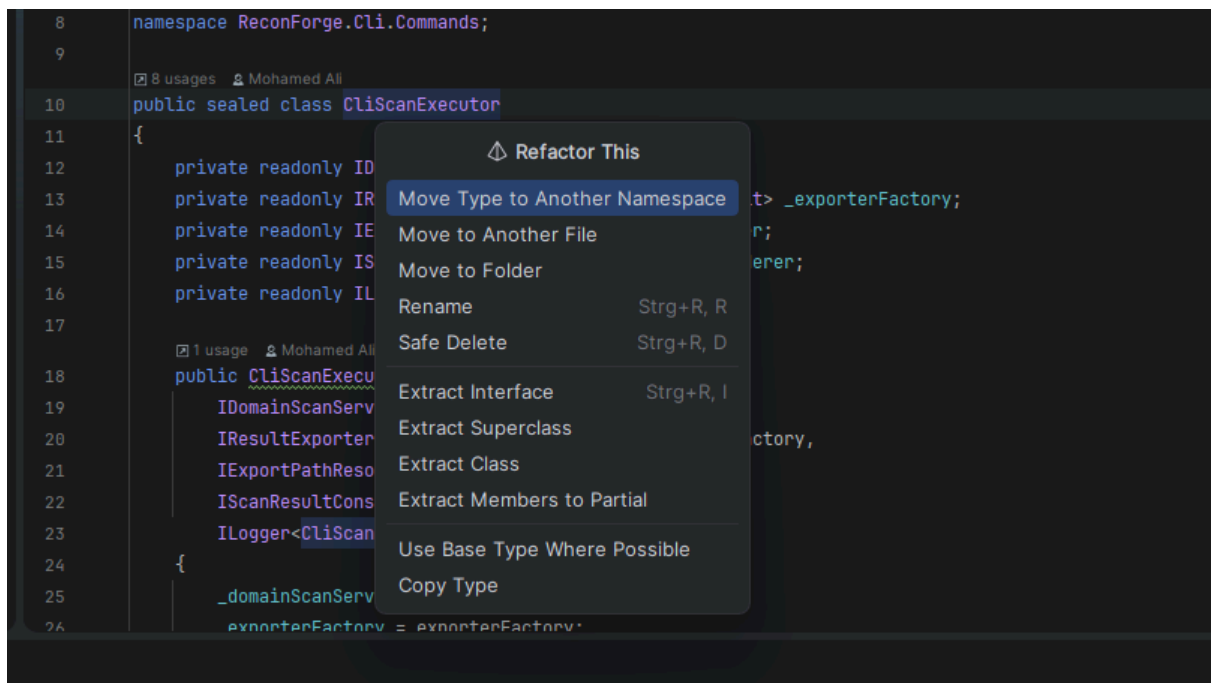
Die verfügbaren Refactorings hängen davon ab, an welcher Stelle im Code sich der Cursor befindet. Wenn man sich zum Beispiel auf einer Klasse befindet und **Refactor This** auswählt, zeigt Rider unter anderem folgende Optionen an:

- Move Type to Another Namespace
- Move to Another File
- Move to Folder
- Rename
- Safe Delete
- Extract Interface
- Extract Superclass
- Extract Class
- Extract Members to Partial

- Use Base Type Where Possible
- Copy Type

Dadurch können Klassen, Methoden und andere Codebestandteile umbenannt, verschoben, ausgelagert oder sicher entfernt werden. Rider zeigt dabei kontextabhängig nur die Refactorings an, die an der aktuellen Stelle im Code sinnvoll sind.





Eigenes Refactoring-Beispiel ohne AI

Für mein eigenes Refactoring-Beispiel habe ich die Klasse `PortScannerService` ausgewählt. In dieser Klasse befindet sich die Methode `ScanAsync`. Beim Durchlesen des Codes ist mir aufgefallen, dass diese Methode relativ lang ist und mehrere Aufgaben gleichzeitig übernimmt.

Die Methode bereitet IP-Adressen vor, erstellt die Portliste, prüft einzelne Ports, behandelt Fehler, erzeugt Ergebnisse und schreibt Log-Ausgaben. Dadurch ist die Methode zwar funktional, aber nicht besonders übersichtlich.

Deshalb habe ich mich für das Fowler-Refactoring **Extract Method** entschieden. Dieses Refactoring passt gut, weil einzelne zusammenhängende Codebereiche aus einer langen Methode in eigene kleinere Methoden ausgelagert werden können.

Das Refactoring wurde manuell mit **JetBrains Rider** durchgeführt. Ich habe dafür keine AI verwendet.

Vorher

Vor dem Refactoring befand sich die Logik direkt in der Methode `ScanAsync`. Dadurch war die Methode länger und schwerer zu lesen.

```
public async Task<IReadOnlyList<PortScanResult>> ScanAsync(
    IEnumerable<ReconForgeIpAddress> resolvedIpAdresse
```

```

s,
    IEnumerable<int>? ports = null,
    int timeoutMs = 1000,
    CancellationToken cancellationToken = default)
{
    cancellationToken.ThrowIfCancellationRequested();

    var targetIpAddresses = NormalizeIpAddresses(resolvedIpAddresses);
    var warnings = new List<string>();
    var portsToCheck = BuildPortList(ports, warnings);
    var results = new List<PortScanResult>();

    _logger.LogInformation("Port Scan started.");
    _logger.LogInformation("Target IP addresses received: {TargetIpAddressCount}", targetIpAddresses.Count);
    _logger.LogInformation("Port list loaded: {PortCount}", portsToCheck.Count);

    if (targetIpAddresses.Count == 0)
    {
        _logger.LogWarning("Port Scan completed with no IP addresses to scan.");
        return results;
    }

    if (portsToCheck.Count == 0)
    {
        _logger.LogWarning("Port Scan completed with no valid ports to check.");
        return targetIpAddresses
            .Select(ipAddress => CreateNoPortsResult(ipAddress, warnings))
            .ToArray();
    }

    foreach (var ipAddress in targetIpAddresses)
    {

```

```

        cancellationToken.ThrowIfCancellationRequested
    );

    var startedAt = DateTimeOffset.UtcNow;
    var checkedPorts = new List<Port>();
    var errors = new List<string>();
    var resultWarnings = new List<string>(warning
s);

    foreach (var portNumber in portsToCheck)
    {
        cancellationToken.ThrowIfCancellationReques
ted();

        _logger.LogInformation("Port check started:
{IpAddress}:{Port}", ipAddress, portNumber);

        try
        {
            var check = await _connectionChecker.Ch
eckAsync(

                ipAddress,
                portNumber,
                timeoutMs,
                cancellationToken);

            var port = new Port(
                portNumber,
                TcpProtocol,
                GetServiceName(portNumber),
                check.State,
                DateTimeOffset.UtcNow,
                check.ResponseTimeMs);

            checkedPorts.Add(port);
            LogPortState(ipAddress, port, check.Err
orMessage);

```

```

        if (check.State == PortState.Error)
        {
            errors.Add($"{ipAddress}:{portNumber}: {check.ErrorMessage ?? "Unknown error"}");
        }
    }
    catch (Exception exception) when (exception is not OperationCanceledException)
    {
        _logger.LogError(exception, "Unexpected exception during port check for {IpAddress}:{Port}", ipAddress, portNumber);
        errors.Add($"{ipAddress}:{portNumber}:{exception.Message}");
        checkedPorts.Add(new Port(
            portNumber,
            TcpProtocol,
            GetServiceName(portNumber),
            PortState.Error,
            DateTimeOffset.UtcNow,
            ResponseTimeMs: null));
    }
}

var status = DetermineStatus(checkedPorts, errors);
var message = status switch
{
    PortScanStatus.Success => "Port scan completed and at least one open port was found.",
    PortScanStatus.PartialSuccess => "Port scan completed with errors.",
    PortScanStatus.Failed => "Port scan failed for this target.",
    _ => "Port scan completed with no open ports."
};

```

```

        results.Add(new PortScanResult(
            ipAddress,
            startedAt,
            DateTimeOffset.UtcNow,
            status,
            message,
            checkedPorts,
            errors,
            resultWarnings));
    }

    var openPortCount = results.Sum(result => result.OpenPorts.Count);

    _logger.LogInformation("Port Scan completed.");
    _logger.LogInformation("Open ports found: {OpenPortCount}", openPortCount);

    return results;
}

```

Nachher

Nach dem Refactoring wurden mehrere Teilaufgaben in eigene Methoden ausgelagert. Dadurch ist `ScanAsync` kürzer und zeigt besser den allgemeinen Ablauf des Port Scans.

```

private async Task<PortScanResult> ScanIpAddressAsync(
    string ipAddress,
    IReadOnlyList<int> portsToCheck,
    IReadOnlyList<string> warnings,
    int timeoutMs,
    CancellationToken cancellationToken)
{
    var startedAt = DateTimeOffset.UtcNow;
    var checkedPorts = new List<Port>();
    var errors = new List<string>();

    foreach (var portNumber in portsToCheck)

```

```

    {
        cancellationToken.ThrowIfCancellationRequested();

        var checkResult = await CheckPortAsync(
            ipAddress,
            portNumber,
            timeoutMs,
            cancellationToken);

        checkedPorts.Add(checkResult.Port);

        if (!string.IsNullOrEmpty(checkResult.ErrorMessage))
        {
            errors.Add(checkResult.ErrorMessage);
        }
    }

    return CreatePortScanResult(
        ipAddress,
        startedAt,
        checkedPorts,
        errors,
        warnings);
}

private async Task<CheckedPortResult> CheckPortAsync(
    string ipAddress,
    int portNumber,
    int timeoutMs,
    CancellationToken cancellationToken)
{
    _logger.LogInformation("Port check started: {IpAddress}:{Port}", ipAddress, portNumber);

    try
    {
        var check = await _connectionChecker.CheckAsync(

```



```

        ipAddress,
        portNumber,
        timeoutMs,
        cancellationToken);

    var port = CreatePort(portNumber, check);

    LogPortState(ipAddress, port, check.ErrorMessage);

    var errorMessage = check.State == PortState.Error
        ? $"{ipAddress}:{portNumber}: {check.ErrorMessage}"
        : null;

    return new CheckedPortResult(port, errorMessage);
}
catch (Exception exception) when (exception is not OperationCanceledException)
{
    _logger.LogError(
        exception,
        "Unexpected exception during port check for {IpAddress}:{Port}",
        ipAddress,
        portNumber);

    var port = CreateErrorPort(portNumber);
    var errorMessage = $"{ipAddress}:{portNumber}: {exception.Message}";

    return new CheckedPortResult(port, errorMessage);
}

private static Port CreatePort(int portNumber, TcpConnectionCheckResult check)
{
    return new Port(

```

```

        portNumber,
        TcpProtocol,
        GetServiceName(portNumber),
        check.State,
        DateTimeOffset.UtcNow,
        check.ResponseTimeMs);
    }

    private static Port CreateErrorPort(int portNumber)
    {
        return new Port(
            portNumber,
            TcpProtocol,
            GetServiceName(portNumber),
            PortState.Error,
            DateTimeOffset.UtcNow,
            ResponseTimeMs: null);
    }

    private static PortScanResult CreatePortScanResult(
        string ipAddress,
        DateTimeOffset startedAt,
        IReadOnlyList<Port> checkedPorts,
        IReadOnlyList<string> errors,
        IReadOnlyList<string> warnings)
    {
        var status = DetermineStatus(checkedPorts, errors);
        var message = GetPortScanMessage(status);

        return new PortScanResult(
            ipAddress,
            startedAt,
            DateTimeOffset.UtcNow,
            status,
            message,
            checkedPorts,
            errors,
            warnings);
    }

```

```

}

private static string GetPortScanMessage(PortScanStatus sta
tus)
{
    return status switch
    {
        PortScanStatus.Success => "Port scan completed and
at least one open port was found.",
        PortScanStatus.PartialSuccess => "Port scan complet
ed with errors.",
        PortScanStatus.Failed => "Port scan failed for this
target.",
        _ => "Port scan completed with no open ports."
    };
}

private static IReadOnlyList<PortScanResult> CreateNoValidP
ortsResults(
    IReadOnlyList<string> targetIpAddresses,
    IReadOnlyList<string> warnings)
{
    return targetIpAddresses
        .Select(ipAddress => CreateNoPortsResult(ipAddress,
warnings))
        .ToArray();
}

private void LogPortScanStarted(
    IReadOnlyList<string> targetIpAddresses,
    IReadOnlyList<int> portsToCheck)
{
    _logger.LogInformation("Port Scan started.");
    _logger.LogInformation("Target IP addresses received:
{TargetIpAddressCount}", targetIpAddresses.Count);
    _logger.LogInformation("Port list loaded: {PortCount}",
portsToCheck.Count);
}

```

```

private void LogPortScanCompleted(IReadOnlyList<PortScanResult> results)
{
    var openPortCount = results.Sum(result => result.OpenPorts.Count);

    _logger.LogInformation("Port Scan completed.");
    _logger.LogInformation("Open ports found: {OpenPortCount}", openPortCount);
}

private sealed record CheckedPortResult(
    Port Port,
    string? ErrorMessage);

```

Verwendetes Fowler-Refactoring

Verwendetes Fowler-Refactoring: **Extract Method**

Ausgelagerte Methoden

Beim Refactoring wurden mehrere kleinere Methoden erstellt, zum Beispiel:

```

ScanIpAddressAsync
CheckPortAsync
CreatePortScanResult
CreatePort
CreateErrorPort
GetPortScanMessage
CreateNoValidPortsResults
LogPortScanStarted
LogPortScanCompleted

```

Zusätzlich wurde ein kleines internes Record `CheckedPortResult` verwendet. Damit kann `CheckPortAsync` neben dem geprüften Port auch eine mögliche Fehlermeldung zurückgeben.

Ergebnis

Durch das Refactoring ist die Methode `ScanAsync` übersichtlicher geworden. Die Methode zeigt jetzt hauptsächlich den Ablauf des Port Scans. Die Details

befinden sich in kleineren Methoden mit klaren Namen.

Das Verhalten des Programms wurde dabei nicht verändert. Es wurde nur die Struktur des Codes verbessert. Dadurch ist der Code besser lesbar, einfacher wartbar und leichter zu testen.

Refactoring-Beispiel mit Generativer AI

Für das zweite Refactoring-Beispiel habe ich eine Generative AI verwendet. Als Beispiel wurde die Methode `ExecuteAsync` aus der Klasse `CliScanExecutor` ausgewählt.

Diese Methode war für ein Refactoring geeignet, weil sie mehrere Aufgaben in einer Methode kombiniert hat, zum Beispiel Logging, Validierung, Scan-Ausführung, Terminalausgabe und optionalen Export.

Ich habe die Methode der Generativen AI gezeigt und sie gebeten, ein Refactoring nach Martin Fowler vorzuschlagen. Dabei sollte vor allem **Extract Method** verwendet werden. Außerdem habe ich angegeben, dass keine fachliche Logik verändert und keine neue Funktion hinzugefügt werden soll.

Die AI hat vorgeschlagen, mehrere kleinere Methoden zu erstellen, zum Beispiel `LogApplicationStart`, `ValidateScanTarget`, `TryResolveExportFormat`, `RunScanAndRenderResultAsync` und `ExportScanResultAsync`.

Dadurch wurde `ExecuteAsync` deutlich übersichtlicher. Die Methode zeigt nach dem Refactoring stärker den Hauptablauf, während einzelne Details in eigene Methoden ausgelagert wurden.

Das Refactoring hat nicht komplett ohne Nacharbeit funktioniert. Ich musste den Vorschlag prüfen, besonders bei `async` / `await`, Rückgabewerten und Fehlercodes. Trotzdem war die Unterstützung hilfreich, weil die AI eine sinnvolle Struktur und passende Methodennamen vorgeschlagen hat.

Am Ende blieb das Verhalten der Methode gleich. Es wurde keine neue Funktion hinzugefügt, sondern nur die Struktur des Codes verbessert.

Prompt:

Ich möchte für meine Softwaretechnik-Abgabe ein Refactoring-Beispiel mit Generativer AI durchführen.

Bitte refaktoriere die folgende Methode aus meinem Projekt ReconForge.
Die Methode heißt `ExecuteAsync` und befindet sich in der Klasse `CliScanExecutor`.

Ich möchte das Fowler-Refactoring Extract Method anwenden.

Bitte beachte:

- Die fachliche Logik darf nicht verändert werden.
- Es sollen keine neuen Features entstehen.
- Das Verhalten soll gleich bleiben.
- Die Methode soll besser lesbar werden.
- Große zusammenhängende Codebereiche sollen in private Hilfsmethoden ausgelagert werden.
- Die neuen Methodennamen sollen verständlich sein.
- async/await und die Rückgabewerte müssen korrekt bleiben.

Hier ist die Methode:

[komplette ExecuteAsync-Methode]

Bitte gib mir eine refaktorierte Version der Methode und der neu extrahierten Hilfsmethoden.

Vorher:

```
public async Task<int> ExecuteAsync(RootCommand.Settings
s settings, CancellationToken cancellationToken = default)
{
    _logger.LogInformation("Application started.");

    if (settings.Verbose)
    {
        _logger.LogInformation("Verbose logging enable
d.");
    }

    if (settings.ScanTarget is null)
    {
        _logger.LogWarning("Validation failed: scan tar
get missing.");
        AnsiConsole.MarkupLine("[red]Error:[/] Provide
a domain with [bold]--scan[/], for example [bold]--scan exa
mple.com[/].");
        return 1;
    }

    _logger.LogInformation("Scan option received.");
    _logger.LogInformation("Domain received: {Domain}",
settings.ScanTarget);
```

```

        if (!settings.Export)
        {
            WarnAboutIgnoredExportOptions(settings);
        }

        var exportFormat = ExportFormat.Json;
        if (settings.Export
            && !ExportFormatParser.TryParse(settings.Format, out exportFormat, out var formatError))
        {
            _logger.LogWarning("Validation failed: unsupported export format {Format}", settings.Format);
            AnsiConsole.MarkupLine("[red]Error:[/] {0}", Markup.Escape(formatError));
            return 1;
        }

        try
        {
            _logger.LogInformation("Scan started for {Domain}", settings.ScanTarget);
            _logger.LogInformation("Scan service called for {Domain}", settings.ScanTarget);

            var result = await _domainScanService.ScanAsync(
                settings.ScanTarget, cancellationToken);

            _logger.LogInformation("Terminal rendering started.");
            _resultRenderer.Render(result, settings.Verbose);
            _logger.LogInformation("Terminal rendering completed.");

            if (result.Status != DomainScanStatus.Success)
            {
                return 1;
            }
        }
    }
}

```

```

    }

    if (!settings.Export)
    {
        _logger.LogInformation("Export skipped because it was not requested.");
        _logger.LogInformation("Command completed.");
        return 0;
    }

    _logger.LogInformation("Export requested.");

    var pathResolution = _exportPathResolver.Resolve(
        result,
        exportFormat,
        settings.OutputPath);

    if (!pathResolution.Success || string.IsNullOrWhiteSpace(pathResolution.OutputPath))
    {
        var message = pathResolution.ErrorMessage ?? "Unable to resolve export output path.";
        _logger.LogWarning("Export path resolution failed: {Reason}", message);
        _logger.LogWarning("Export failed.");
        AnsiConsole.MarkupLine("[red]Error: [/] {0}", Markup.Escape(message));
        return 1;
    }

    _logger.LogInformation("Selected export format: {ExportFormat}", exportFormat);
    _logger.LogInformation("Output path selected: {OutputPath}", pathResolution.OutputPath);

    var exporter = _exporterFactory.Create(exportFo

```



```

rmat);

        var exportResult = await exporter.ExportAsync(
            result,
            pathResolution.OutputPath,
            cancellationToken);

        if (!exportResult.Success)
        {
            var message = exportResult.ErrorMessage ??
exportResult.Message;
            _logger.LogWarning("Export failed: {Reason}", message);
            AnsiConsole.MarkupLine("[red]Export failed:
[/] {0}", Markup.Escape(message));
            return 1;
        }

        AnsiConsole.MarkupLine("[green]Results exported
successfully to:[/] {0}", Markup.Escape(exportResult.Output
Path));

        _logger.LogInformation("Export completed.");
        _logger.LogInformation("Command completed.");
        return 0;
    }
    catch (Exception exception)
    {
        _logger.LogError(exception, "Unexpected excepti
on.");
        AnsiConsole.MarkupLine("[red]Unexpected error:
[/] {0}", Markup.Escape(exception.Message));
        return 1;
    }
}

```

Nachher:

```

public async Task<int> ExecuteAsync(RootCommand.Settings settings, CancellationToken cancellationToken = default)
{
    LogApplicationStart(settings);

    if (!ValidateScanTarget(settings))
    {
        return 1;
    }

    LogScanTarget(settings);

    if (!TryResolveExportFormat(settings, out var exportFormat))
    {
        return 1;
    }

    try
    {
        var result = await RunScanAndRenderResultAsync(settings, cancellationToken);

        if (result.Status != DomainScanStatus.Success)
        {
            return 1;
        }

        if (!settings.Export)
        {
            LogExportSkipped();
            return 0;
        }

        return await ExportScanResultAsync(
            result,
            exportFormat,
            settings.OutputPath,

```

```

        cancellationToken);
    }
    catch (Exception exception)
    {
        return HandleUnexpectedException(exception);
    }
}

private void LogApplicationStart(RootCommand.Settings settings)
{
    _logger.LogInformation("Application started.");

    if (settings.Verbose)
    {
        _logger.LogInformation("Verbose logging enabled.");
    }
}

private bool ValidateScanTarget(RootCommand.Settings settings)
{
    if (settings.ScanTarget is not null)
    {
        return true;
    }

    _logger.LogWarning("Validation failed: scan target missing.");
    AnsiConsole.MarkupLine("[red]Error:[/] Provide a domain with [bold]--scan[/], for example [bold]--scan example.com[/].");
    return false;
}

private void LogScanTarget(RootCommand.Settings settings)
{
    _logger.LogInformation("Scan option received.");
}

```

```

        _logger.LogInformation("Domain received: {Domain}", settings.ScanTarget);
    }

    private bool TryResolveExportFormat(
        RootCommand.Settings settings,
        out ExportFormat exportFormat)
    {
        exportFormat = ExportFormat.Json;

        if (!settings.Export)
        {
            WarnAboutIgnoredExportOptions(settings);
            return true;
        }

        if (ExportFormatParser.TryParse(settings.Format, out exportFormat, out var formatError))
        {
            return true;
        }

        _logger.LogWarning("Validation failed: unsupported export format {Format}", settings.Format);
        AnsiConsole.MarkupLine("[red]Error:[/] {0}", Markup.Escape(formatError));
        return false;
    }

    private async Task<DomainScanResult> RunScanAndRenderResultAsync(
        RootCommand.Settings settings,
        CancellationToken cancellationToken)
    {
        _logger.LogInformation("Scan started for {Domain}", settings.ScanTarget);
        _logger.LogInformation("Scan service called for {Domain}", settings.ScanTarget);
    }

```

```

        var result = await _domainScanService.ScanAsync(settings.ScanTarget, cancellationToken);

        _logger.LogInformation("Terminal rendering started.");
        _resultRenderer.Render(result, settings.Verbose);
        _logger.LogInformation("Terminal rendering completed.");

        return result;
    }

    private void LogExportSkipped()
    {
        _logger.LogInformation("Export skipped because it was not requested.");
        _logger.LogInformation("Command completed.");
    }

    private async Task<int> ExportScanResultAsync(
        DomainScanResult result,
        ExportFormat exportFormat,
        string? outputPath,
        CancellationToken cancellationToken)
    {
        _logger.LogInformation("Export requested.");

        var pathResolution = _exportPathResolver.Resolve(
            result,
            exportFormat,
            outputPath);

        if (!pathResolution.Success || string.IsNullOrEmpty(pathResolution.OutputPath))
        {
            var message = pathResolution.ErrorMessage ?? "Unable to resolve export output path.";
            _logger.LogWarning("Export path resolution failed:

```

```

{Reason}", message);
    _logger.LogWarning("Export failed.");
    AnsiConsole.MarkupLine("[red]Error:[/] {0}", Markup
p.Escape(message));
    return 1;
}

_logger.LogInformation("Selected export format: {Export
Format}", exportFormat);
_logger.LogInformation("Output path selected: {OutputPa
th}", pathResolution.OutputPath);

var exporter = _exporterFactory.Create(exportFormat);
var exportResult = await exporter.ExportAsync(
    result,
    pathResolution.OutputPath,
    cancellationTokens);

if (!exportResult.Success)
{
    var message = exportResult.ErrorMessage ?? exportRe
sult.Message;
    _logger.LogWarning("Export failed: {Reason}", messa
ge);
    AnsiConsole.MarkupLine("[red]Export failed:[/]
{0}", Markup.Escape(message));
    return 1;
}

AnsiConsole.MarkupLine("[green]Results exported success
fully to:[/] {0}", Markup.Escape(exportResult.OutputPath));

_logger.LogInformation("Export completed.");
_logger.LogInformation("Command completed.");
return 0;
}

private int HandleUnexpectedException(Exception exception)

```

```
{  
    _logger.LogError(exception, "Unexpected exception.");  
    AnsiConsole.MarkupLine("[red]Unexpected error:[/] {0}",  
Markup.Escape(exception.Message));  
    return 1;  
}
```

Abschließend habe ich die Tests ausgeführt, um sicherzustellen, dass alles in Ordnung ist.

Solution ▾

ReconForge - 6 projects

src - 5 projects

ReconForge.Cli

Dependencies

Commands

C# CliScanExecu

C# RootComman

DependencyInje

Presentation

C# Program.cs

ReconForge.Core

Dependencies

Abstractions

Exporting

Models

Validation

ReconForge.Export

ReconForge.Infrast

ReconForge.Scanni

tests - 1 project

Scratches and Consoles

C# CliScanExecutor.cs

C# IPortScannerService.cs

C# PortScannerService.cs

⚠

Rider detects naming conventions in opened solutions and updates settings a...

Configure

Disable

Close

```

10  public sealed class CliScanExecutor
32  public async Task<int> ExecuteAsync(RootCommand.Settings settings, CancellationToken cancellation
35
36      if (!ValidateScanTarget(settings))
37      {
38          return 1;
39      }
40
41      LogScanTarget(settings);
42
43      if (!TryResolveExportFormat(settings, out var exportFormat))
44      {
45          return 1;
46      }
47
48      try
49      {
50          var result = await RunScanAndRenderResultAsync(settings, cancellationToken);
51
52          if (result.Status != DomainScanStatus.Success)
53          {
54              return 1;
55          }
56
57          if (!settings.Export)
58          {
59              LogExportSkipped();
60              return 0;
61          }
62
63          return await ExportScanResultAsync(
64              result,
65              exportFormat,
66              settings.OutputPath,
67              cancellationToken);
68      }
69      catch (Exception exception)
70      {
71          return HandleUnexpectedException(exception);
72      }
73
74
75      private void LogApplicationStart(RootCommand.Settings settings)

```

Terminal Local x

Windows PowerShell

Copyright (C) Microsoft Corporation. Alle Rechte vorbehalten.

Installieren Sie die neueste PowerShell für neue Funktionen und Verbesserungen! <https://aka.ms/PSWindows>

```

PS C:\Users\Mohamed\Documents\ReconForge> dotnet test
Wiederherstellung abgeschlossen (0,8s)
  ReconForge.Core net10.0 Erfolgreich (0,1s) → src\ReconForge.Core\bin\Debug\net10.0\ReconForge.Core.dll
  ReconForge.Export net10.0 Erfolgreich (0,1s) → src\ReconForge.Export\bin\Debug\net10.0\ReconForge.Export.dll
  ReconForge.Scanning net10.0 Erfolgreich (0,2s) → src\ReconForge.Scanning\bin\Debug\net10.0\ReconForge.Scanning.dll
  ReconForge.Infrastructure net10.0 Erfolgreich (0,2s) → src\ReconForge.Infrastructure\bin\Debug\net10.0\ReconForge.Infrast
  ructure.dll
  ReconForge.Cli net10.0 Erfolgreich (0,3s) → src\ReconForge.Cli\bin\Debug\net10.0\ReconForge.Cli.dll
  ReconForge.Tests net10.0 Erfolgreich (0,2s) → tests\ReconForge.Tests\bin\Debug\net10.0\ReconForge.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.7)
[xUnit.net 00:00:00.92] Discovering: ReconForge.Tests
[xUnit.net 00:00:01.23] Discovered: ReconForge.Tests
[xUnit.net 00:00:01.29] Starting: ReconForge.Tests
Warning: --output requires --export and will be ignored.
Error: Unsupported export format 'pdf'. Supported formats are: json, csv, xml,
html, yaml.
Results exported successfully to:
C:\Users\Mohamed\AppData\Local\Temp\reconforge-cb376fc6ecd2491cb19f75b74670c6c0.
json
Warning: --format is ignored unless --export is provided.
Warning: --output requires --export and will be ignored.
Error: Provide a domain with --scan, for example --scan example.com.
Warning: --format is ignored unless --export is provided.
[xUnit.net 00:00:03.08] Finished: ReconForge.Tests
  ReconForge.Tests Test net10.0 Erfolgreich (4,7s)

Testzusammenfassung: insgesamt: 115; fehlgeschlagen: 0; erfolgreich: 115; Übersprungen: 0; Dauer: 4,7 Sek.
Erstellen von Erfolgreich in 6,6s

PS C:\Users\Mohamed\Documents\ReconForge>

```



```
PS C:\Users\Mohamed\Documents\ReconForge> dotnet run --project src/ReconForge.Cli -- --scan hackthissite.org
```



Modular Reconnaissance CLI Tool
Fast signals. Structured results. Modular workflow.
For educational and authorized testing only.

ReconForge Scan Results

Target Information

Field	Value
Original input	hackthissite.org
Normalized host	hackthissite.org
Root domain	hackthissite.org
Scan start time	2026-06-14 15:13:26Z
Scan end time	2026-06-14 15:16:49Z
Duration	202,67 s
Status	Success

Subdomains

Subdomain	Source	Discovered At
www.hackthissite.org	DefaultPrefix	2026-06-14 15:13:26Z
mail.hackthissite.org	DefaultPrefix	2026-06-14 15:13:26Z
api.hackthissite.org	DefaultPrefix	2026-06-14 15:13:26Z
dev.hackthissite.org	DefaultPrefix	2026-06-14 15:13:26Z
test.hackthissite.org	DefaultPrefix	2026-06-14 15:13:26Z
staging.hackthissite.org	DefaultPrefix	2026-06-14 15:13:26Z
admin.hackthissite.org	DefaultPrefix	2026-06-14 15:13:26Z

Resolved IP Addresses

Host	IP Address	Address Family	Is Private	Is Loopback
hackthissite.org	137.74.187.101	InterNetwork	No	No
hackthissite.org	137.74.187.104	InterNetwork	No	No
hackthissite.org	137.74.187.102	InterNetwork	No	No
hackthissite.org	137.74.187.103	InterNetwork	No	No
hackthissite.org	137.74.187.100	InterNetwork	No	No
api.hackthissite.org	2001:41d0:8:ccd8:137:74:187:116	InterNetworkV6	No	No
api.hackthissite.org	2001:41d0:8:ccd8:137:74:187:118	InterNetworkV6	No	No
api.hackthissite.org	2001:41d0:8:ccd8:137:74:187:119	InterNetworkV6	No	No
api.hackthissite.org	2001:41d0:8:ccd8:137:74:187:117	InterNetworkV6	No	No
api.hackthissite.org	2001:41d0:8:ccd8:137:74:187:115	InterNetworkV6	No	No
api.hackthissite.org	137.74.187.116	InterNetwork	No	No
api.hackthissite.org	137.74.187.118	InterNetwork	No	No
api.hackthissite.org	137.74.187.117	InterNetwork	No	No
api.hackthissite.org	137.74.187.115	InterNetwork	No	No
api.hackthissite.org	137.74.187.119	InterNetwork	No	No

Open Ports

IP Address	Port	Protocol	State	Service Name	Response Time
137.74.187.101	80	TCP	Open	HTTP	42 ms
137.74.187.101	443	TCP	Open	HTTPS	34 ms
137.74.187.104	80	TCP	Open	HTTP	43 ms
137.74.187.104	443	TCP	Open	HTTPS	38 ms
137.74.187.102	80	TCP	Open	HTTP	37 ms
137.74.187.102	443	TCP	Open	HTTPS	36 ms
137.74.187.103	80	TCP	Open	HTTP	37 ms
137.74.187.103	443	TCP	Open	HTTPS	41 ms
137.74.187.100	80	TCP	Open	HTTP	44 ms
137.74.187.100	443	TCP	Open	HTTPS	44 ms

Summary

Field	Value
Subdomains discovered	7
Hosts resolved	2
IP addresses resolved	15
Ports checked	210
Open ports	10
Timed out ports	200
Failed host resolutions	5
Real warnings	0
Errors	0

Warnings and Errors

! 5 hosts could not be resolved. Use --verbose for details.

```
PS C:\Users\Mohamed\Documents\ReconForge> █
```

